

MLFF VRAM1 + PERF-P4: Workload-Correct Capacity and Bounded DATA6 Pipeline

mdstats development

2026-08-15

1. Status

Release: mdstats 0.20.186a0

Architecture: revision 53

VRAM1 authority: schema-changing capacity evidence

PERF-P4 authority: execution-only

Accelerator qualification: deferred to FINAL-GPU1

VRAM1 corrects DATA6 memory planning so calibration measures the workload that production actually runs. PERF-P4 adds bounded overlap between CPU graph preparation, accelerator evaluation, and durable shard persistence. Neither gate may change descriptors, predictions, frame order, selection authority, or downstream decisions.

2. VRAM1 capacity evidence

`MaceBatchCapacityCalibration.v2` replaces descriptor-only v1 capacity evidence for new campaigns. Historical v1 records remain readable and retain their original meaning.

Each v2 record binds:

- descriptor signature and checkpoint identity;
- device and explicit workload mode: `descriptor_only`, `prediction_only`, or `combined_evaluate`;
- deterministic stress-frame identities;
- requested, probed, successful, and recommended batch sizes;
- per-probe elapsed time and structures/s;
- baseline, peak, and post-probe CUDA allocated/reserved bytes;
- driver-visible free/total memory;
- descriptor, graph, and serialized-prediction host-memory estimates;
- configured device-memory budget, absolute reserve, fractional occupancy limit, and throughput tolerance; and
- post-cleanup allocator and driver-visible memory state.

The calibration corpus is selected deterministically by realized CPU graph footprint, then atom count, then immutable structure digest. This deliberately avoids a first- N sampling bias.

2.1 Safe throughput-aware batch rule

Let $\mathcal{P}$ be the successful probes and $\mathcal{S} \subseteq \mathcal{P}$ those satisfying every declared headroom constraint. With measured throughput $q(B)$ and tolerance ϵ , choose

$$B^* = \min \left\{ B \in \mathcal{S} : q(B) \geq (1 - \epsilon) \max_{b \in \mathcal{S}} q(b) \right\}.$$

The generated tolerance is $\epsilon = 0.05$. Choosing the smallest near-best batch avoids consuming memory for negligible throughput gain.

The safe set requires both

$$R_{\text{peak}}(B) \leq f M_{\text{total}}$$

and

$$M_{\text{free,after}}(B) \geq M_{\text{reserve}},$$

plus any stricter campaign device budget. The generated defaults are $f = 0.80$ and $M_{\text{reserve}} = 4 \text{ GiB}$; they are execution policy, not scientific constants.

PyTorch distinguishes tensor allocation from caching-allocator reservation, and its CUDA APIs expose both process allocator statistics and driver-visible free/total memory.¹ VRAM1 records both views rather than inferring device pressure from one counter.

2.2 Cleanup and live re-clamp

Calibration releases probe-local state, synchronizes CUDA, performs Python garbage collection when needed, calls `torch.cuda.empty_cache()` once, synchronizes again, and then re-reads live memory. `empty_cache()` is not called between ordinary successful batches. PyTorch documents that it releases unoccupied cached memory but does not increase the memory available to PyTorch itself; it is therefore a cleanup boundary, not a capacity oracle.²

Immediately before DATA6, the calibrated prior is clamped against fresh live memory:

$$M_{\text{inc}} = \min(M_{\text{free}} - M_{\text{reserve}}, f M_{\text{total}} - M_{\text{reserved}}, M_{\text{campaign}}),$$

where absent campaign budget terms are omitted. A stale calibration can never authorize a batch larger than current memory permits.

2.3 Durable OOM learning

A CUDA OOM with adaptive batching enabled performs

$$B_{n+1} = \max(1, \lfloor B_n/2 \rfloor)$$

without advancing scientific frame order. After a reduced batch succeeds, DATA6 persists `Data6RuntimeBatchCap.v1` containing the safe/rejected pair and OOM count. The cap identity binds checkpoint, descriptor policy, device, dtype, workload mode, and capacity-calibration digest. A changed identity invalidates the cap.

3. PERF-P4 bounded pipeline

For native MACE combined evaluation, PERF-P4 permits:

```
CPU graph build for batch n+1
    ||
accelerator evaluation for batch n
    ||
CPU shard persistence for batch n-1
```

The implementation has two independent single-worker executors:

1. a producer that creates the next immutable CPU graph batch; and
2. a persistence worker that writes descriptor/prediction shards.

¹PyTorch CUDA memory APIs: <https://docs.pytorch.org/docs/stable/cuda.html>; `torch.cuda.mem_get_info`: https://docs.pytorch.org/docs/stable/generated/torch.cuda.memory.mem_get_info.html.

²PyTorch, `torch.cuda.memory.empty_cache`: https://docs.pytorch.org/docs/main/generated/torch.cuda.memory.empty_cache.html.

The model itself is evaluated only on the main execution path. Graph prefetch is enabled only when the provider exposes the native prepared-batch interface. Non-native providers retain the existing path.

Python’s `ThreadPoolExecutor` is used as a bounded asynchronous call interface; it does not change scientific ordering.³

3.1 Memory admission

At most one active inference batch, one prepared CPU graph batch, and one or two persistence chunks may be resident. Host admission uses the conservative per-structure quantity

$$H_{\text{frame}} = H_{\text{graph}} + H_{\text{descriptor}} + H_{\text{prediction}},$$

with

$$B_{\text{host}} = \left\lfloor \frac{M_{\text{RAM,budget}}}{H_{\text{frame}}(2 + Q_{\text{persist}})} \right\rfloor.$$

The actual execution batch cannot exceed this bound.

3.2 Durability and determinism

Scientific order is the immutable DATA6 plan order. Worker completion order does not become evidence.

- Payload files are written and fsynced before their records can be committed.
- Persistence futures are drained in submission order.
- Append-only journal events are appended only after payload persistence succeeds.
- Journal flush/checkpoint semantics remain the existing recovery authority.
- Final manifests are reconstructed from ordered committed records.
- Synchronous execution remains an exact fallback.

An OOM invalidates any prefetched batch created for the rejected batch size; retry begins at the same scientific cursor with the reduced batch.

4. Transfer policy

PERF-P4 does **not** enable page-locked/nonblocking transfer merely because CUDA is available. PyTorch documents that asynchronous transfer behavior depends on pinned memory, synchronization, transfer direction, and workload; its own guidance recommends benchmarking the concrete case rather than assuming manual pinning is faster.⁴

Pinned-memory and nonblocking-copy variants are therefore final-GPU experiments. If they do not improve throughput under the same memory bound, the synchronous transfer path remains authoritative execution policy.

5. CPU/reference qualification

The development host is CPU-only, so it cannot close VRAM1/PERF-P4 accelerator acceptance. The following implementation evidence is qualified now:

- v1 calibration compatibility and v2 round trip;
- throughput/headroom decision logic;
- fresh-live-memory and host-queue clamps through deterministic mocked-resource tests;
- durable OOM cap reuse only for matching identity;

³Python documentation, `concurrent.futures`: <https://docs.python.org/3/library/concurrent.futures.html>.

⁴PyTorch, *A guide on good usage of `non_blocking` and `pin_memory()`*: https://docs.pytorch.org/tutorials/intermediate/pinmem_nonblock.html.

- exact synchronous/pipeline DATA6 scientific equality;
- exact prepared/direct combined-evaluation equality on the supplied MACE-MH-1/`omat_pbe` and MACE-MPA-0-medium/`default` models under CPU/e3nn; and
- bounded persistence queue validation.

The 44-frame CPU control-plane benchmark uses 15 alternating repetitions per mode. Both modes produce scientific signature

`c07e1bb049703c0b160b88b18bfa0c6c0c788198cf21a6d0454ef9c19c689a96`.

Median wall times are 72.89 ms synchronous and 76.32 ms pipelined. The pipeline is therefore 4.72% slower on this tiny CPU fixture. This is recorded as orchestration overhead, not hidden or reinterpreted as a speedup. The accelerator pipeline remains optional until final-GPU evidence demonstrates benefit.

6. Deferred FINAL-GPU1 acceptance

The final release-matched CUDA qualification must still demonstrate:

1. workload-specific v2 calibration under the locked MH-1 and MPA-0 foundations;
2. forced batch 1, calibrated batch, and deliberate OOM-backoff scientific agreement;
3. bounded peak allocated/reserved VRAM under the declared absolute/fractional reserve;
4. fresh-memory re-clamping after calibration cleanup;
5. matching-identity restart reuse of the learned OOM cap;
6. pipeline versus synchronous throughput and memory pressure;
7. pinned/nonblocking variants only if they are benchmarked and memory-accounted; and
8. no change in descriptors, predictions, target authority, or downstream decisions beyond already-authorized model numerical tolerances.

Until those items pass, the gate state is **implemented / CPU-control-plane qualified / accelerator qualification pending**.

7. Next implementation gate

The requested no-interruption workflow advances to **PERF-P5** for CPU/control-plane implementation. E3NN-BASELINE and VRAM1/PERF-P4 accelerator performance qualification remain final-release execution obligations under FINAL-GPU1.